

Java: acceso a bases de datos

6.3.2024

Enrique Krause Buedo

1 Introducción

6.3.2024

Enrique Krause Buedo

Conceptos fundamentales

Tabla: una tabla es un componente de una base de datos que almacena los datos en registros (filas) y en campos (columnas). Se suele agrupar la información por categoría a nivel de una tabla. Por ejemplo teniendo la tabla de los Clientes, de los Productos o de los Pedidos.

Registro: el registro es el conjunto de datos relativos a un elemento en una tabla. Los registros son los equivalentes al nivel lógico de la fila de una tabla. Por ejemplo, un registro de la tabla Clientes contiene las características de un cliente particular. También se llaman "tuplas"

Campo: un registro se compone de varios campos. Cada campo de un registro contiene una única información en el registro. Por ejemplo un registro Cliente puede contener los campos CódigoCliente, Apellido, nombre

Conceptos fundamentales

Clave primaria: se utiliza una clave primaria para identificar unívocamente una fila de una tabla. La clave primaria es un campo o una combinación de campos cuyo valor es único en la tabla. Por ejemplo, el campo CodigoCliente es la clave primaria de la tabla Cliente. No pueden existir dos clientes con el mismo código.

Clave foránea: una clave foránea representa uno o varios campos de una tabla que hacen referencia a los campos de la clave primaria de otra tabla. Las claves foráneas indican cómo se relacionan las tablas entre sí.

Relación: una relación es una asociación establecida entre campos comunes en dos tablas. Una relación puede ser de uno a uno, uno a varios y varios a varios. Gracias a las relaciones, los resultados de peticiones pueden contener datos procedentes de varias tablas. Una relación de uno a varios entre la tabla Cliente y la tabla Pedido permite a una petición retornar pedidos correspondientes a un cliente.

Select

El lenguaje SQL permite especificar los registros que se han de extraer, así como el orden en el cual deseamos extraerlos. Se puede crear una instrucción SOL que extrae datos de varias tablas simultáneamente o se puede crear una instrucción que extrae solamente un registro específico. Se utiliza la instrucción SELECT para devolver unos campos específicos de una o de varias tablas de la base de datos.

WHERE ... IN Puede buscar todos los clientes que vivan en Francia o España.

```
SELECT * FROM Cliente WHERE País IN ('Francia','España')
```

WHERE ... Between Puede utilizarse con numéricos o fechas

WHERE ... LIKE "%es%" los % actúan como reemplazo de secuencia de otras letras de3lante atrás o en ambos casos

ORDER BY retorna registros ordenados por columnas, puede usarse en común con limit y offset

Inserción

La creación de los registros en una tabla se hace con el comando INSERT INTO. Se ha de indicar la tabla en la cual deseamos insertar una línea, la lista de los campos en los cuales deseamos especificar un valor y, por último, la lista de los valores correspondientes. La sintaxis completa es, por lo tanto, la siguiente:

```
Insert INTO Cliente (códigoCliente,apellido,nombre) VALUES (1000,  
'García','Pedro')
```

En el momento de la inserción de este nuevo cliente, solo se insertará en la tabla el código, el apellido y el nombre. Los demás campos tomarán el valor null. Si no se indica la lista de campos, la instrucción INSERT exige que se especifique un valor para cada campo de la tabla. Por lo tanto, es necesario utilizar la palabra clave NULL para indicar que de un campo particular no hay información. Si la tabla Cliente se compone de cinco campos (Códigocliente, Apellido, Nombre, Dirección, País) se puede sintetizar la instrucción anterior con la sintaxis siguiente:

```
INSERT INTO Cliente VALUES (1000, 'García','Pedro',NULL, NULL)
```

6.3.2024

Supresión de datos

La instrucción DELETE FROM permite suprimir uno o varios registros de una tabla. Debemos proporcionar como mínimo el nombre de la tabla. Si no se indica nada más, se suprimen todas las filas de la tabla. En general, se añade una cláusula WHERE que limita el alcance de la supresión. El comando siguiente, solo suprime un registro particular:

```
DELETE FROM Cliente WHERE CódigoCliente=1000
```

Por supuesto, el lenguaje SQL es mucho más completo.

Acceso a una base de datos desde Java

Cuando se desea manipular una base de datos a partir de un lenguaje de programación, se dispone de dos soluciones:

- Comunicarse directamente con la base de datos.
- Utilizar una capa de software que asegure el diálogo con la base de datos.

La primera solución comporta varios requisitos.

Debe dominar perfectamente la programación de red.

- También debe conocer en detalle el protocolo utilizado por la base de datos.
- Este tipo de desarrollo suele ser muy largo y lleno de trabas. Por ejemplo, ¿puede acceder a las especificaciones del protocolo?
- Deberá empezar de nuevo todo su trabajo si cambia de tipo de base de datos ya que por supuesto los protocolos no son compatibles de una base de datos a otra. O incluso peor, de una versión a otra de una misma base de datos.

Acceso a una base de datos desde Java

Por supuesto, la segunda solución es preferible y es la que los diseñadores de han elegido. Para ello, desarrollaron la **API JDBC (Java DataBase Connectivity)** para el acceso a una base de datos. Precisamente, la APIJDBC se commpone de dos partes. La primera parte, contenida en el paquete `java.sql`, se compone esencialmente de interfaces. Estas interfaces están implementadas por los drivers JDBC. Estos drivers (o puentes) no están desarrollados por Oracle sino, en general, por la empresa diseñadora de la base de datos. Efectivamente, es el propio fabricante quien domina mejor la técnica para comunicarse con su base de datos. Existen cuatro tipos de drivers JDBC con caraterísticas y rendimiento diferentes.

Tipo 1: Driver JDBC-ODBC Este tipo de puente no es específico de una base de datos concreta, sino que traduce simplemente las instrucciones `jdbc` en instrucciones ODBC. A continuación, es el driver `odbc` quien asegura la comunicación con la base de datos. Esta solución solo ofrece resultados mediocres porque dependen del número de capas de software utilizadas. Las funcionalidades `jdbc` también están limitadas por las de la capa ODEC. Este tipo de controlador no está disponible a partir de la versión 8 de Java. Oracle recomienda utilizar el controlador específico de la base de datos con la que desee trabajar. Está, generalmente, disponible en el sitio del fabricante de la base de datos.

Acceso a una base de datos desde Java

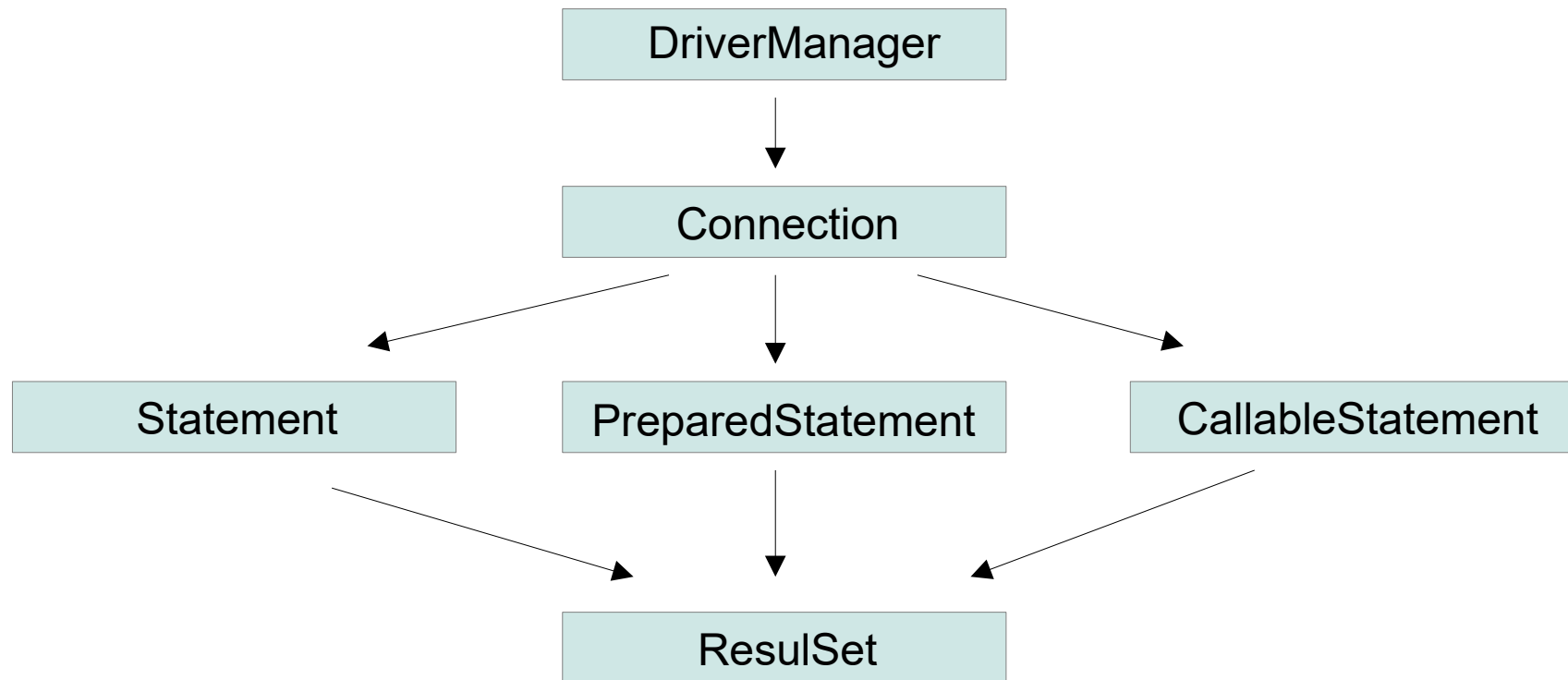
Tipo 2: Driver nativo Este tipo de puente no está escrito completamente en Java. La parte de este driver escrita en Java efectúa sencillamente llamadas hacia funciones del driver nativo. Se efectúan estas llamadas mediante la API JNL (Java Native Interface). De la misma forma que para los puentes de tipo 1, hace falta una traducción entre el código Java y la base de datos. Sin embargo. Este tipo sigue siendo más eficaz que los drivers JDBC-ODBC.

Tipo 3: Driver que utiliza un servidor intermediario Este puente de protocolo de red, desarrollado completamente en Java no se comunica directamente con la base de datos, sino que se dirige a un servidor intermediario. El diálogo entre el driver y el servidor intermediario es estándar sea cual sea el tipo de base de datos. A continuación, es este servidor intermediario quien transmite las peticiones utilizando un protocolo específico para la base de datos.

Tipo 4: Driver completamente escrito en Java Este tipo de driver representa la solución ideal, ya que no hay intermediario alguno. El driver transmite directamente las peticiones a la base de datos utilizando el protocolo propio de la base de datos. La mayoría de los actuales son de este tipo.

Presentación de JDBC

La API JDBC proporciona un conjunto de clases y sobre todo de interfaces que permiten la manipulación de una base de datos. Estos elementos representan todo lo necesario para acceder a los datos desde una aplicación de Java



Presentación de JDBC

La clase **DriverManager** es nuestro punto de partida. Es ella quien asegura el vínculo con el driver. A través de ella, podemos obtener una conexión hacia la base de datos. Está representada por una instancia de clase que implementa la interfaz **Connection**. Á continuación, se utiliza esta conexión para transmitir instrucciones a la base de datos. Las peticiones simples se ejecutan mediante la interfaz **Statement**; las peticiones con parámetros, con la interfaz **PreparedStatement**, y los procedimientos almacenados, con la interfaz **CallableStatement**.

Los eventuales registros seleccionados por la instrucción SQL se recogen y devuelven en forma de un elemento **ResultSet**. Vamos a detallar estas diferentes etapas

Carga del driver

La primera etapa indispensable es obtener el driver JDBC adaptado a su base de datos. En general, este controlador está disponible para su descarga en el sitio del diseñador de la base de datos. Para nuestros ejemplos, utilizaremos el controlador facilitado por Oracle para acceder a un servidor de base de datos MySQL. La versión 8.0 del driver (llamado Connector/J) es necesario para utilizar MySQL 8.0.

El driver en Maven:

<https://mvnrepository.com/artifact/mysql/mysql-connector-java/8.0.27>

De modo que este archivo esté accesible en la compilación, hay que copiarlo en el directorio lib del proyecto, y luego con el boton derecho en el proyecto→ build

Path→Configure Build path

pestaña Librerías→ botón Add Jars: seleccionar el jar descargado. ok.

el directorio lib contiene físicamente el controlador, que debe estar en classPath. El driver se carga directamente durante la ejecución Hay que consultar la documentación de cada driver de manera que sepamos el nombre de la clase que tenemos que llamar desde java. En un try catch, este caso

```
Class.forName("com.mysql.cj.jdbc.Driver");
```

Establecer y manipular la conexión

El método **getConnection** de la clase **DriverManager**, se encarga de conectar y propone tres soluciones. El método recibe, en todas las versiones un String de una URL. Es específico en cada driver, hay que consultar documentación de manera que sepamos la sintaxis adecuada en cada caso. El principio, es estándar `jdbc:nombreDelProtocolo...` cuyo nombre es específico en cada driver, y permite a `getConnection` identificarlo. El resto es específico. En general contiene los datos de servidor, base de datos... En el caso de MySQL:

```
Jdbc:mysql://direiP/NombreBaseDatos?User=NomUser&password=passUser;
```

En caso de éxito el método `getConnection` retorna una instancia de una clase que implementa `Connection`

ANÁLISIS Y PROGRAMACIÓN EN JAVA - IFCD004PO

```
import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.SQLException;
public class TestConnexion {
public static void main(String[] args) {
    String url = "jdbc:mysql://localhost/0_tienda";
    String user = "root";
    String password = "root";
    try {
        Connection cnx = DriverManager.getConnection(url, user, password);
        System.out.println("Estado de la conexión :");
        System.out.println(cnx.isClosed() ? "cerrada" : "abierta");
    } catch (SQLException e) {
        System.out.println("Un error se ha producido durante la conexión a
la base de datos");
        e.printStackTrace();
    }
}
```

6.3.2024

Enrique Krause Buedo

Establecer la conexión

Para que este código se compile, es necesario cargar el módulo java.sql, porque no se carga de manera predeterminada. Para hacer esto, simplemente modifique el archivo module-info. java de este modo:

```
module Nombreproyecto {  
    requires java.sql;  
  
}
```

La ejecución de este programa muestra el siguiente mensaje:

Estado de la conexión:

abierta

Tenéis que tener la BD o_tienda

Además el puerto debe ser el por defecto 3306

Manipular la conexión

Una conexión queda abierta desde el momento en que se crea. Por lo tanto, no existe un método específico que permita abrir una conexión. En cambio, se puede cerrar una conexión llamando al método `close`. Es importante hacer esto cuando la conexión, ya no sea útil para liberar los recursos comprometidos. Después del cierre de una conexión, ya no es posible usarla y debe volver a crearse para poder utilizarla de nuevo.

La interfaz `Connection` era la interfaz `AutoCloseable`. Por lo tanto, no es necesario usar el método `close` si el desarrollador hace uso de la instrucción `try with resources`. El siguiente ejemplo muestra el uso de una instrucción como esta:

ANÁLISIS Y PROGRAMACIÓN EN JAVA - IFCD004PO

```
import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.SQLException;
public class TestConnexionTryWithResources {
    public static void main(String[] args) {
        String url = "jdbc:mysql://localhost/0_tienda";
        String user = "root";
        String password = "root";
        //Uso de la instrucción try with resources
        try(Connection cnx = DriverManager.getConnection(url, user, password)) {
            System.out.println("Estado de la conexión :");
            System.out.println(cnx.isClosed() ? "cerrada" : "abierta");
        } //cierre automática de la conexión
        catch (SQLException e) {
            System.out.println("Un error se ha producido durante la conexión a la
base de datos");
            e.printStackTrace();
        }
    }
}
```

6.3.2024

Enrique Krause Buedo

Manipular la conexión

La función **isClosed** permite comprobar si una conexión está esta función devuelve un booleano igual a false, se puede pensar con razón que la conexión está abierta y que permite dialogar, por lo tanto, con la base de datos. En realidad, esto no es siempre así. La conexión puede estar en un estado intermedio: no está cerrada, pero no puede utilizarse en transferir instrucciones hacia el servidor. Para comprobar la disponibilidad de la conexión, puede utilizar el método **isValid**. Este método comprueba realmente la disponibilidad de la conexión intentando enviar una instrucción SQL y verificando que obtiene una respuesta de parte del servidores. Este método no está implementado en todos los drivers; en ese caso, produce una excepción del tipo `java.lang.UnsupportedOperationException`, o un error del tipo `java.lang.AbstractMethodError`. Este código muestra el uso de este método

Hay que llamarlo antes del catch del código anterior

```
System.out.println(cnx.isClosed() ? "cerrada" : "abierta");  
  
    testEsValido(cnx);  
  
} catch (SQLException e) {
```

ANÁLISIS Y PROGRAMACIÓN EN JAVA - IFCD004PO

```
public static void testEsValido(Connection cnx) {  
    boolean esValido;  
    try {  
        esValido = cnx.isValid(0);  
        if (esValido) {  
            System.out.println("la conexión es válida");  
        } else {  
            System.out.println("La conexión no es válida");  
        }  
    } catch (Exception e) {  
        System.out.println("El método isValid no se soporta por el  
controlador");  
        e.printStackTrace();  
    }  
}
```

6.3.2024

Enrique Krause Buedo

Manipular la conexión

Si únicamente se efectúan operaciones de lectura en la base de datos, podemos optimizar la comunicación precisando que la conexión está en solo lectura. El método **setReadOnly** permite modificar esta configuración de la conexión. A continuación, se puede comprobar el estado usando el método `isReadOnly`. En algunos drivers, esta funcionalidad no está implementada y el método **setReadOnly** no surte ningún efecto. La función siguiente verifica si esta funcionalidad está disponible para la conexión que recibe como parámetro.

hay que llamarla también antes del catch

```
testReadingSolo (cnx) ;
```

ANÁLISIS Y PROGRAMACIÓN EN JAVA - IFCD004PO

```
public static void testReadingSolo(Connection cnx) {  
    boolean estado;  
    try {  
        estado = cnx.isReadOnly();  
        cnx.setReadOnly(!estado);  
        if (cnx.isReadOnly() != estado) {  
            System.out.println("el modo solo lectura se soporta por este  
controlador");  
        } else {  
            System.out.println("el modo solo lectura no se soporta por este  
controlador");  
        }  
        cnx.setReadOnly(estado);  
    } catch (SQLException e) {  
        e.printStackTrace();  
    }  
}
```

6.3.2024

Enrique Krause Buedo

Manipular la conexión

Durante la ejecución de una instrucción SQL, el servidor puede detectar problemas y, por lo tanto, generar advertencias. Es posible recuperar las advertencias mediante el método **getWarnings** de la conexión. devuelve un objeto **SQLWarning** que representa el problema encontrado por el servidor. Si encuentra varios problemas, genera objetos **SQLWarning** encadenados entre sí. El método **getNextWarning** permite obtener el elemento siguiente o null si se ha alcanzado el final de la lista. Se puede vaciar con el método **clearWarnings**. La función siguiente visualiza todos los avisos recibidos por la conexión.

```
public static void visualizacionWarnings(Connection cnx) {  
    SQLWarning advertencia;  
    Try { advertencia = cnx.getWarnings();  
    if (advertencia == null) {  
        System.out.println("No hay advertencia");  
    } else { while (advertencia != null) {  
        System.out.println(advertencia.getMessage());  
        System.out.println(advertencia.getSQLState());  
        System.out.println(advertencia.getErrorCode());  
        advertencia = advertencia.getNextWarning();  
    }  
    cnx.clearWarnings();  
} catch (SQLException e) {e.printStackTrace();}  
}
```

Manipular la conexión

En general, en el momento de la creación de la cadena de conexión, uno de sus parámetros determina el nombre de la base de datos con la que queremos establecer una conexión. La modificación del nombre de la base de datos a la que estamos conectados se lleva a cabo con el método **setCatalog**, al cual hay que proporcionar el nombre de otra base de datos presente en el mismo servidor. Por supuesto, hace falta que la cuenta con la que se abrió la sesión “isponga de permisos de acceso suficientes para esta base de datos. Cabe señalar que con este método cambiamos de base de datos, pero la conexión se sigue refiriendo al mismo servidor. No hay ningún medio para cambiar de servidor sin crear una nueva conexión.

```
public static void cambiarDeBaseDeDatos(Connection cnx) {  
    try {  
        System.out.println("base actual : " + cnx.getCatalog());  
        System.out.println("cambio de base de datos");  
        cnx.setCatalog("mi_negocio");  
        System.out.println("base actual : " + cnx.getCatalog());  
    } catch (SQLException e) {  
        System.out.println("Problema en la manipulación de las bases de datos en  
el servidor");  
        e.printStackTrace();  
    }  
}
```

Estado de la conexión :
abierta
la conexión es válida
base actual : 0_tienda
cambio de base de datos
base actual : mi_negocio

Manipular la conexión

Es posible también obtener la estructura de la base de datos mediante el método **getMetaData**. Este método devuelve un objeto de tipo **DataBaseMetaData** que facilita numerosa información relativa a la estructura de la base de datos. La función siguiente muestra un análisis rápido de esta información.

```
public static void infosBase(Connection cnx) {
    ResultSet rs;
    DatabaseMetaData dbmd;
    try {
        dbmd = cnx.getMetaData();
        System.out.println("tipo de base : " + dbmd.getDatabaseProductName());
        System.out.println("versión: " + dbmd.getDatabaseProductVersion());
        System.out.println("nombre del controlador : " + dbmd.getDriverName());
        System.out.println("versión del controlador : " + dbmd.getDriverVersion());
        System.out.println("nombre del usuario : " + dbmd.getUserName());
        System.out.println("url de conexión : " + dbmd.getURL());
        rs = dbmd.getTables(null, null, "%", null);
        System.out.println("estructura de la base");
        System.out.println("base\ttesquema\tnombre table\tttype table");
        while (rs.next()) {
            for (int i = 1; i <= 4; i++) {
                System.out.print(rs.getString(i) + "\t");
            }
            System.out.println();
        }
        rs.close();
        rs = dbmd.getProcedures(null, null, "%");
        System.out.println("los procedimientos almacenados");
        System.out.println("base\ttesquema\tnombre procedimiento");
        while (rs.next()) {
            for (int i = 1; i <= 3; i++) {
                System.out.print(rs.getString(i) + "\t");
            }
            System.out.println();
        }
        rs.close();
    } catch (SQLException e) {
        e.printStackTrace();
    }
}
```

6.3.2024

Enrique Krause Buedo

Manipular la conexión

Puede observar en este ejemplo el uso de un objeto de tipo ResultSet permite leer los conjuntos de resultados obtenidos en el servidor.

Todos los métodos presentados hasta ahora pueden ser de ayuda algún día, pero la meta principal de una conexión es permitir la ejecución de instrucciones SQL. Son los objetos de tipo **Connection** los que van a facilitar los objetos necesarios para la ejecución de estas instrucciones. Se pueden ejecutar tres tipos de instrucciones SQL:

- Las peticiones simples.
- Las peticiones precompiladas.
- Los procedimientos almacenados.

Manipular la conexión

A cada uno de estos tipos le corresponde una clase de la API JDBC:

1 Statement para las peticiones simples.

2 PreparedStatement para las peticiones precompiladas.

3 CallableStatement para los procedimientos almacenados.

Para terminar, se puede obtener una instancia de estas clases a partir de un método diferente de un objeto de tipo **Connection**:

createStatement para los objetos de tipo Statement.

prepareStatement para los objetos de tipo PreparedStatement.

prepareCall para los objetos de tipo CallableStatement.

Ejecución de instrucciones SQL

Antes de la ejecución de una instrucción SQL, debemos elegir el tipo de objeto JDBC más apropiado.

1 Instrucciones con el objeto de tipo Statement

Un objeto de este tipo se obtiene mediante el método `createStatement` de la conexión. JDBC cuenta con dos versiones de este método: la primera no recibe ningún parámetro. En este caso, si se utiliza el objeto `Statement` de manera que se pueda ejecutar una instrucción SQL que genera un juego de registros `select` este únicamente estará en solo lectura y con un desplazamiento de los datos hacia adelante únicamente.

Los datos presentes en este juego de registros no podrán modificarse y solo se podrá hacer el recorrido del juego de registros desde el primero hasta el último. La segunda versión permite elegir las características del juego de registros generado. Recibe dos parámetros. El primero determina el tipo del juego de registros

Se definen las constantes siguientes de la clase `ResultSet`

1 Instrucciones con el objeto de tipo Statement

Se definen las constantes siguientes de la clase ResultSet

- `ResultSet.TYPE_FORWARD_ONLY`: el juego de registros será de desplazamiento hacia delante únicamente.
- `ResultSet.TYPE_SCROLL_INSENSITIVE`: el juego de registros podrá recorrerse en ambos sentidos, pero será insensible a los cambios en la base de datos por otros usuarios.
- `ResultSet.TYPE_SCROLL_SENSITIVE`: el juego de registros podrá recorrerse en ambos sentidos y será sensible a los cambios realizados en la base de datos por otros usuarios.

El segundo argumento determina las posibilidades de modificación de datos contenidos en el juego de los registros. Las dos constantes siguientes están definidas:

- `ResultSet.CONCUR_READ_ONLY`: los registros son de solo lectura
- `ResultSet.CONCUR_UPDATABLE`: los registros pueden modificarse dentro del juego de registros.

1 Instrucciones con el objeto de tipo Statement

Este objeto Statement es el más elemental que permite la ejecución de instrucciones SQL. Puede encargarse de la ejecución de cualquier instrucción SQL. Por lo tanto, puede ejecutar tanto instrucciones DDL (Data Definition Language) como instrucciones DML (Data Manipulation Language). Solo hace falta elegir en este objeto el método mejor adaptado para la ejecución del código SQL. Debería seleccionar un método u otro en función del tipo de resultado que devuelve la instrucción SQL. Hay cuatro métodos disponibles:

- **public boolean execute (String sql)**: este método permite ejecutar cualquier instrucción SQL. El boolean devuelto por este método indica si se ha generado algún juego de registros (true) O si simplemente la instrucción modificó registros en la base de datos (false). Si se genera un juego de registros, puede obtenerse mediante el método **getResultSet**. Si ha habido registros modificados en la base de datos, el método **getUpdateCount** devuelve el número de registros modificados. La función siguiente ilustra el uso de estos métodos.

1 Instrucciones con el objeto de tipo Statement

```
import java.io.*;
import java.sql.*;
public class TestStatement {
    public static void main(String[] args) {
        try (Connection cnx = getConnexion();
            Statement stm = cnx.createStatement(ResultSet.TYPE_SCROLL_SENSITIVE,
            ResultSet.CONCUR_READ_ONLY)) {
            BufferedReader br = new BufferedReader(new InputStreamReader(System.in));
            System.out.println("rellenar su instrucción SQL :");
            String consulta = br.readLine();
            boolean resultado = stm.execute(consulta);
            if (resultado) {
                System.out.println("su instrucción ha generado un juego de registros");
                //Recuperación del juego de resultado
                ResultSet rs = stm.getResultSet();
                //Posicionamiento en el último registro
                rs.last();
                //Reading de número de la línea
                System.out.println("contiene " + rs.getRow() + " registros");
            } else {
                System.out.println("su instrucción ha modificado de las registros en la base");
                System.out.println("número de registros modificados : " + stm.getUpdateCount());
            }
        } catch (SQLException e) {
            System.out.println("su instrucción no ha funcionado correctamente");
        } catch (IOException e) {
            e.printStackTrace();
        }
    }
    private static Connection getConnexion() throws SQLException {
        String url = "jdbc:mysql://localhost/mi_negocio";
        String user = "root";
        String password = "root";
        return DriverManager.getConnection(url, user, password);
    }
}
```

rellenar su instrucción SQL :
SELECT * FROM productos where idproducto =5;
su instrucción ha generado un juego de registros
contiene 1 registros

rellenar su instrucción SQL :
INSERT INTO `mi\_negocio`.`productos`
(`idproducto`,`prod\_descripcion`,
`prod\_marca`,`prod\_modelo`,
`prod\_familia`,`prod\_precio`,
`prod\_fecha\_precio`,`prod\_stock`) VALUES
(`24`,`Disco externo 2 TB`,`Seagate`,
`2t`,`Disco externo`,`75.00`,`2021-05-
15`,`5`);
su instrucción ha modificado de las
registros en la base
número de registros modificados :1

6.3.2024

Enrique Krause Buedo

1 Instrucciones con el objeto de tipo Statement

- **public Resultset executeQuery (String sentencia):** este método fue diseñado especialmente para la ejecución de instrucciones select. El juego de los registros está disponible directamente mediante el valor devuelto por la función.
- **public int executeUpdate (String sentencia):** este método se adapta perfectamente a la ejecución de instrucciones que modifican el contenido de la base de datos, como son las instrucciones insert, update, delete. El valor entero devuelto por esta función indica el número de registros afectados por la modificación.
- **public int[] executeBatch():** este método permite e conjunto de instrucciones SQL por lote. El lote de instrucciones que ejecutar debe prepararse previamente mediante los métodos addBatch y clearBatch. El primero recibe como parámetro una cadena de que representa una instrucción SQL que hay que añadir al lote. El segundo permite reinicializar el lote de instrucciones. Este método es muy práctico para modificaciones en varias tablas relacionadas. Este podría ser, por ejemplo, el caso en una aplicación de gestión comercial con una tabla para las solicitudes y otra para las líneas de detalle de cada solicitud. La supresión de un comando debe causar la supresión de todas las líneas correspondientes.

1 Instrucciones con el objeto de tipo Statement

Otros métodos van a intervenir en el comportamiento del Statement:

- **public void setQueryTimeout (int duracion):** este indica la duración máxima asignada para la ejecución antes de producir una excepción.
- **public void close ():** cuando un objeto Statement ya no es útil en una aplicación, es preferible cerrarlo explícitamente invocando este método. Esto provoca la liberación de todos los recursos usados por este objeto. El cierre de un objeto Statement provoca también el cierre del juego de registros asociado (ResultSet). Al igual que la interfaz Connection, la interfaz Statement hereda de la interfaz AutoCloseable. Por lo tanto, es posible (como muestran los ejemplos) inicializar este tipo de objeto en una instrucción try with resources, para provocar su cierre automático al final bloque try.
- **public void setMaxRows (int número):** este método número de líneas de los juegos de registros generados por este objeto Statement. Si una instrucción SQL genera un juego de registros que contiene más líneas, las líneas de más simplemente se ignoran (sin más información).

1 Instrucciones con el objeto de tipo Statement

Otros métodos van a intervenir en el comportamiento del Statement:

- **public void setMaxFieldSize (int tamaño):** este método limita el tamaño de algunos tipos de campos en el juego de registros. Los tipos de campos afectados son los campos de una base de datos que pueden tener un tamaño variable, como por ejemplo los campos compuestos por cadenas de caracteres o datos binarios. Los datos de más simplemente se ignoran. Los campos afectados pueden ser de hecho inutilizables en la aplicación.
- **public void setFetchSize(int numLíneas):** cuando una instrucción SQL genera un juego de registros, los datos correspondientes se transfieren desde el servidor de base de datos hacia la memoria de la aplicación de Java. Este traslado se efectúa por bloques según el uso de los datos. Este método indica al driver el número de líneas de cada bloque transferido desde la base de datos hacia la aplicación.

1 Instrucciones con el objeto de tipo Statement

Otros métodos van a intervenir en el comportamiento del Statement:

- **public boolean getMoreResults():** si se utiliza el método execute para ejecutar varias instrucciones SQL, por ejemplo dos instrucciones select, en este caso, se generan dos juegos de registros. El primero se obtiene con el método getResultSet. El segundo solo será accesible después de haber llamado el método getMoreResults. Esta función permite desplazar el puntero sobre el resultado siguiente y devuelve un booleano igual a true si el resultado siguiente es un juego de registros. En tal caso, dicho juego de registros también es accesible mediante el método getResultSet. Si la función getMoreResults devuelve un booleano false, es que el resultado siguiente no es un juego de registros o que sencillamente no hay ningún resultado siguiente. Para resolver la duda, hay que invocar la función getUpdateCount y comprobar el valor devuelto por esta función. Si este valor es igual a -1, es que no hay ningún resultado siguiente; en caso contrario, el valor devuelto representa el número de registros modificados en la database

1 Instrucciones con el objeto de tipo Statement

```
import java.io.*;
import java.sql.*;
public class TestStatementGetMoreResults {
    public static void main(String[] args) {
        boolean resultado;
        ResultSet rs;
        try (Connection cnx = getConnexion();
            Statement stm = cnx.createStatement(ResultSet.TYPE_SCROLL_SENSITIVE,
ResultSet.CONCUR_READ_ONLY)) {
            BufferedReader br = new BufferedReader(new InputStreamReader(System.in));
            System.out.println("rellenar sus instrucciones SQL separadas por ; :");
            String consulta = br.readLine();
            resultado = stm.execute(consulta);
            int i = 1;
            // tratamiento del resultado generado por la primera instrucción
            if (resultado) {
                System.out.println("su instrucción N° " + i + " ha generado un juego de registros");
                rs = stm.getResultSet();
                rs.last();
                System.out.println("contiene " + rs.getRow() + " registros");
            } else {
                System.out.println("su instrucción N° " + i + " ha modificado los registros en la
base");
                System.out.println("número de registros modificados :" + stm.getUpdateCount());
            }
            i++;
            // movimiento del puntero sobre el eventual resultado siguiente
            resultado = stm.getMoreResults();
            // bucle muestra que haya un resultado de tipo de juego de registro -> resultado==true
            // o que haya un resultado de tipo número de registros modificados -> getUpdateCount != -1
        }
    }
}
```

1 Instrucciones con el objeto de tipo Statement

```
while (resultado || stm.getUpdateCount() != -1) {
    if (resultado) {
        System.out.println("su instrucción N° " + i + " ha generado un juego de
registros");
        rs = stm.getResultSet();
        rs.last();
        System.out.println("contiene " + rs.getRow() + " registros");
    } else {
        System.out.println("su instrucción N° " + i + " ha modificado los registros en
la base");
        System.out.println("número de registros modificados :" + stm.getUpdateCount());
    }
    i++;
    // movimiento del puntero sobre el eventual resultado siguiente
    resultado = stm.getMoreResults();
}
} catch (SQLException e) {
    System.out.println("su instruccion no ha funcionado correctamente");
    e.printStackTrace();

} catch (IOException e) {
    e.printStackTrace();
}
}
private static Connection getConnexion() throws SQLException {
    String url = "jdbc:mysql://localhost/mi_negocio";
    String user = "root";
    String password = "root";
    return DriverManager.getConnection(url, user, password);
}
}
```

rellenar sus instrucciones SQL separadas por ; :
SELECT * FROM productos where prod_precio > 100.0 order by prod_precio;
su instrucción N° 1 ha generado un juego de registros
contiene 9 registros

6.3.2024

Enrique Krause Buedo

2 Instrucciones con el objeto tipo PreparedStatement

Ocurre a menudo que hay que ejecutar varias veces una petición SQL con una pequeña modificación entre dos ejecuciones. El ejemplo clásico corresponde a una petición de selección con una restricción.

```
String petition = "select * from PERSONAS where nombre = \"Juan\";
```

El valor sobre el cual recae la restricción suele introducirlo el usuario de la aplicación y en este caso está disponible en una variable. La primera solución que viene a la mente consiste en construir la petición SQL por concateción de varias cadenas de caracteres.

```
String petition = "select * from PERSONAS where nombre = '" + nombreIntroducido +  
"'" ;
```

Esta solución presenta varios inconvenientes:

- La concatenación de cadenas de caracteres consume muchos recursos
- El servidor debe analizar cada vez una nueva petición.
- La sintaxis se va a volver compleja rápidamente si es necesario concatenar varias cadenas. Siempre hay que recordar que al final debemos obtener una instrucción SQL correcta.

2 Instrucciones con el objeto tipo PreparedStatement

Finalmente, la concatenación puede conducir a riesgos de inyecciones SQL que desvirtuarían la función inicial de la consulta. Imagine la búsqueda por identificar a una persona por su nombre de usuario y contraseña. Introducidos en un formulario. Podría ser de la siguiente forma:

```
String petition = "select * from PERSONAS where login = '" + loginIntroducido + '"  
and contrasenia = '" + contraseniaIntroducida + "'";
```

Si el usuario rellena los valores «admin» y «mdp1234», entonces la consulta generada es la siguiente:

```
Select * from PERSONAS where login = 'admin' and contrasenia = 'mdp1234'
```

En consecuencia, si el usuario introduce los valores «admin» y «' or 'a'='a » la consulta adquiere otro aspecto y el comportamiento cambia. Un atacante puede iniciar sesión sin conocer la contraseña del usuario. Esto se llama inyección SQL. El error de seguridad en este ejemplo es interpretar las comillas introducidas por el usuario como un elemento de sintaxis SQL. Debe escapar este carácter para indicar que se tiene que interpretar solo como un carácter de texto.

2 Instrucciones con el objeto tipo PreparedStatement

Los diseñadores de JDBC tienen prevista una solución eficaz para resolver estos inconvenientes. El objeto PreparedStatement aporta una solución eficaz a este problema al permitirnos crear peticiones con parámetros. En este tipo de petición, se sustituyen los parámetros por signos de interrogación. Antes de la ejecución de la petición, hay que facilitar al objeto PreparedStatement los valores que debe utilizar para remplazar los **diferentes signos de interrogación**. Se puede crear un objeto PreparedStatement utilizando el mismo principio que para la creación de un objeto Statement.

El método `prepareStatement` accesible a partir de una conexión devuelve un objeto PreparedStatement. Este método está disponible bajo dos formas. La primera recibe como argumento una cadena de caracteres que representa la petición SQL. **En esta petición, la ubicación de los parámetros debe estar reservada por signos de interrogación**. Si se crea un juego de registros con la ejecución de esta petición, será de solo lectura y de desplazamiento únicamente hacia delante. La segunda forma del método **`prepareStatement`** recibe, además de la cadena de caracteres, un argumento que indica el tipo de juego de registros y otro que determina las posibilidades de modificación de los datos contenidos en el juego de registros

2 Instrucciones con el objeto tipo PreparedStatement

Se pueden utilizar las mismas constantes que para el método createStatement.

```
PreparedStatement pstmt=cnx.prepareStatement ("select + PERSONAS  
where nombre=?", ResultSet.TYPE_SCROLL_SENSITIVE,  
ResultSet.CONCUR_READ_ONLY);
```

Antes de la ejecución de la instrucción SQL, debemos facilitar un valor por cada uno de los signos de interrogación que representan un parámetro. Con este fin, el objeto PreparedStatement dispone de numerosos métodos que permiten asignar un valor a un parámetro. Cada uno de estos corresponde al tipo de datos SQL que se ha de insertar en lugar de cada signo de interrogación. Estos métodos siguen la siguiente nomenclatura: setXxx, donde Xxx representa un tipo de datos SQL. Cada uno de estos métodos como primer argumento un entero que corresponde al rango del parámetro en la instrucción SQL. El primer parámetro se sitúa en el rango 1. El segundo argumento corresponde al valor que se ha de transferir en el parámetro. El tipo de este argumento corresponde por supuesto al tipo de datos SQL.

2 Instrucciones con el objeto tipo PreparedStatement

Por ejemplo, el método `setInt (int indiceParam, int value)` efectúa una conversión del tipo `int java` en tipo `INTEGER SQL` cada proveedor de un controlador JDBC proporciona documentación que enumera las relaciones entre los tipos Java y los tipos SQL. Esta documentación del controlador JDBC está disponible en la siguiente dirección:

<https://dev.mysql.com/doc/connector-j/8.0/en/connector-j-reference-type-conversions.html>

Se conservan los valores almacenados en los parámetros de una ejecución de la instrucción SQL a la siguiente. El método `clearParameters` permite reinicializar el conjunto de los parámetros.

Se pueden utilizar los parámetros en una instrucción SQL en sustitución de unos valores, pero nunca para sustituir el nombre de un campo, y menos de un operador. Por supuesto, la sintaxis siguiente está prohibida

```
PreparedStatement pstmt = cnx.prepareStatement: ("select from PERSONAS where ? = ?",  
ResultSet.TYPE_SCROLL_SENSITIVE, ResultSet.CONCUR_READ_ONLY);  
pstmt.setString(1,"nombre");  
pstmt.setString(2,"Juan");  
rs=pstmt.executeQuery ();
```

2 Instrucciones con el objeto tipo PreparedStatement

Los demás métodos disponibles con un objeto PreparedStatement son idénticos a los definidos para un objeto Statement, ya que la interfaz PreparedStatement hereda directamente de la interfaz Statement. El siguiente ejemplo muestra la lista de productos que responden a la marca proporcionado por el usuario de la aplicación.

```
import java.io.*;
import java.sql.*;
public class TestPreparedStatement {
    public static void main(String[] args) {
        try (Connection cnx = getConnexion());
            PreparedStatement pstmt = cnx.prepareStatement("select * from productos where
prod_marca = ?",
                    ResultSet.TYPE_SCROLL_SENSITIVE, ResultSet.CONCUR_READ_ONLY)) {
                BufferedReader br = new BufferedReader(new InputStreamReader(System.in));
                System.out.println("rellenar el nombre de la marca buscada :");
                String nombre = br.readLine();
                //Aplicar los parámetros
                pstmt.setString(1, nombre);
                //Ejecutar la consulta y obtener el resultado
                ResultSet rs = pstmt.executeQuery();
                //Mientras hay registros
                while (rs.next()) {
                    for (int i = 1; i <= rs.getMetaData().getColumnCount(); i++) {
                        System.out.print(rs.getString(i) + "\t");
                    }
                    System.out.println();
                }
            } catch (SQLException e) {
                e.printStackTrace();
            } catch (IOException e) {
                e.printStackTrace();
            }
        }
    }

    private static Connection getConnexion() throws SQLException {
        String url = "jdbc:mysql://localhost/mi_negocio";
        String user = "root";
        String password = "root";
        return DriverManager.getConnection(url, user, password);
    }
}
```

rellenar el nombre de la marca
buscada :
Seagate
1 Disco externo 1 Tb usb Seagate
FireCuda Disco duro 155.80
2020-04-12 11
6 Disco 4Tb usb 3.0 Seagate
Barracuda Disco duro 315.00 2020-
07-30 6
22 Disco externo 2.5" 4TB USB 3.0
Seagate Expansion Disco externo
92.99 2021-01-12 31
24 Disco externo 2 TB Seagate
2t Disco externo 75.00 2021-
05-15 5

2 Instrucciones con el objeto tipo PreparedStatement

El tipo PreparedStatement también se usa mucho para las operaciones de escritura (insertar, actualizar, eliminar). Dentro del marco de inserción, una funcionalidad muy apreciable hace posible repetir la clave autogenerada por la base de datos (este suele ser el caso de una clave primaria del tipo de incremento automático).

Para recuperar la clave generada automáticamente, debe usar la segunda firma del método prepareStatement del objeto de tipo connection. Este método espera como parámetro una cadena de caracteres que indica si el desarrollador desea recuperar la clave autogenerada por la base de datos. La constante Statement.RETURN_GENERATED_KEYS se usa de modo que se especifica esto.

La ejecución de la consulta parametrizada con el método executeUpdate de la interfaz PreparedStatement permite evaluar un ResultSet que contiene un único registro con la clave autogenerada. El acceso a este ResultSet se hace a través del método **getGeneratedKeys** de la interfaz Statement, de la que deriva la interfaz PreparedStatement. La clave está disponible en la primera columna del único registro de ResultSet. **He creado DB y tabla por este ejemplo**

```
create database pruebasJava;  
create table PERSONAS ( id int auto_increment primary key,  
nombre varchar (30) not null, apellido varchar (30) not null,  
fechaDenacimiento date );
```

2 Instrucciones con el objeto tipo PreparedStatement

```
import java.io.*;
import java.sql.*;
public class TestInsertionViaPreparedStatement {
    public static void main(String[] args) {
        try (Connection cnx = getConnexion();
            //Consulta de insertion con los campos obligatorios
            PreparedStatement pstmt = cnx.prepareStatement("insert into PERSONAS(nombre, apellido)
values(?,?)",
                                Statement.RETURN_GENERATED_KEYS)) {
            BufferedReader br = new BufferedReader(new InputStreamReader(System.in));
            System.out.println("introducir el nombre de la persona a agregar :");
            String nombre = br.readLine();
            System.out.println("introducir el apellido de la persona a agregar :");
            String apellido = br.readLine();
            //Aplicar los parámetros
            pstmt.setString(1, nombre);
            pstmt.setString(2, apellido);
            //Ejecutar la consulta y obtener el resultado
            pstmt.executeUpdate();
            //Recuperar el ResultSet que eventualmente contiene una clave
            ResultSet rsClaveGenerada = pstmt.getGeneratedKeys();
            //Si hay un registro, es el que tiene una clave
            if(rsClaveGenerada.next())
            {
                System.out.println("La clave de la registro generado es " +
rsClaveGenerada.getInt(1));
            }
        } catch (SQLException e) {
            e.printStackTrace();
        } catch (IOException e) {
            e.printStackTrace();
        }
    }
    private static Connection getConnexion() throws SQLException {
        String url = "jdbc:mysql://localhost/pruebasjava";
        String user = "root";
        String password = "root";
        return DriverManager.getConnection(url, user, password);
    }
}
```

introducir el nombre de la
persona a agregar :
Pablo
introducir el apellido de la
persona a agregar :
Marmol
La clave de la registro
generado es 2

6.3.2024

Utilización de los juegos de registros con la interfaz ResultSet

Cuando se ejecuta la instrucción SQL select con el método `executeQuery` de un objeto `Statement`, `PreparedStatement` o `CallableStatement`, este devuelve un objeto `ResultSet`. A través de este objeto `ResultSet` vamos a poder intervenir sobre el juego de registros. Nuestras posibilidades de acción sobre este juego de registros están determinadas por las características de objeto `ResultSet`. Se fijan estas características en el momento de la creación de los objetos `Statement`, `PreparedStatement` o `CallableStatement` en función de los argumentos pasados durante la llamada a los métodos `createStatement`, `prepareStatement` o `prepareCall`.

Utilización de los juegos de registros con la interfaz ResultSet

El primer argumento determina el tipo del juego de registros. Se definen las constantes siguientes:

- `ResultSet.TYPE_FORWARD_ONLY`: el juego de registros se desplazará solo hacia delante.
- `ResultSet.TYPE_SCROLL_INSENSITIVE`: el juego de registros podrá recorrerse en ambos sentidos, pero será insensible a los cambios realizados en la base de datos por otros usuarios.
- `ResultSet.TYPE_SCROLL_SENSITIVE`: el juego de registros podrá recorrerse en ambos sentidos y será sensible a los cambios realizados en la base de datos por otros usuarios.

Utilización de los juegos de registros con la interfaz ResultSet

El segundo argumento determina las posibilidades de modificación de los datos contenidos en el juego de registros. Se definen las dos constantes siguientes:

- ResultSet.CONCUR_READ_ONLY: los registros son de solo lectura.
- ResultSet.CONCUR_UPDATABLE: los registros pueden modificarse en el juego de registros.

Por supuesto, hace falta que las acciones ejecutadas sobre el objeto ResultSet sean compatibles con estas características; en caso contrario, se producirá una excepción. Es posible comprobar las características de un objeto Resultset utilizando los métodos `getType` y `getConcurrency`. El siguiente código muestra las características del ResultSet.

2 Instrucciones con el objeto tipo PreparedStatement

```
private static void mostrarTypeResultSet(ResultSet rs) {  
    try {  
        switch (rs.getType()) {  
            case ResultSet.TYPE_FORWARD_ONLY:  
                System.out.println("el ResultSet solo se desplaza hacia adelante");  
                break;  
            case ResultSet.TYPE_SCROLL_INSENSITIVE:  
                System.out.println("el ResultSet se puede recorrer en ambos sentidos");  
                System.out.println("no es sensible a las modificaciones hechas por los otros usuarios");  
                break;  
            case ResultSet.TYPE_SCROLL_SENSITIVE:  
                System.out.println("el ResultSet se puede recorrer en ambos sentidos");  
                System.out.println("il est sensible a las modificaciones hechas por los otros usuarios");  
                break;  
        }  
        switch (rs.getConcurrency()) {  
            case ResultSet.CONCUR_READ_ONLY:  
                System.out.println("los datos contenidos en el ResultSet son de solo lectura ");  
                break;  
            case ResultSet.CONCUR_UPDATABLE:  
                System.out.println("los datos contenidos en el ResultSet son modificables");  
                break;  
        }  
    } catch (SQLException e) {  
        e.printStackTrace();  
    }  
}
```

Se trata de un
método que se
puede incluir en
el código
anterior

Posicionamiento en un ResultSet

El objeto ResultSet gestiona un puntero de registro que determina sobre qué registro van a intervenir los métodos ejecutados sobre el propio ResultSet. Este registro se llama a veces registro activo o registro en curso. El ResultSet contiene siempre dos registros ficticios que sirven de referencia para el principio del ResultSet (BOF - Beginning Of File) y para el fin del ResultSet (EOF - End Of File). El puntero de registro se puede colocar sobre uno de estos dos registros, pero nunca antes del registro BOF ni después del registro EOF. Estos registros no contienen datos y una operación de lectura o escritura sobre ellos genera una excepción. En el momento de la creación del ResultSet, el puntero se coloca antes del primer registro (BOF). Cuando el conjunto de registros no contiene resultados, necesariamente hay estos registros ficticios.

Posicionamiento en un ResultSet

Existen muchos métodos disponibles para gestionar el puntero de registros:

boolean absolute (int position): desplaza el puntero de sobre el registro especificado. La numeración de los registros empieza en 1. Si el valor del argumento posición es negativo, se efectúa el desplazamiento partiendo del final del Resultset. Si el número de registro no existe, se coloca el puntero sobre el registro BOF si el valor es negativo e inferior al número de registros, o sobre el registro EOF si el valor es positivo y superior al número de registros. Este método devuelve true si el puntero está colocado en un registro válido y false en el caso contrario (BOF o EOF).

boolean relative(int displacement): desplaza el puntero del número de registros especificados por el argumento displacement. Si el valor de este argumento es positivo, el cursor baja en el juego de registros. Y si el valor es negativo, el cursor sube en el juego de registros. Este método devuelve true si el puntero está colocado sobre un registro válido y en el caso contrario (BOF o EOP).

void beforeFirst (): desplaza el puntero de registros antes del primer registro (BOF).

Posicionamiento en un ResultSet

void afterLast (): desplaza el puntero de registros después del último registro (EOF).

boolean first (): desplaza el puntero de registros sobre el primer registro. Este método devuelve true si hay un registro en el ResultSet y false en caso contrario.

boolean last (): desplaza el puntero de registros sobre el último registro. Este método devuelve true si hay un registro en el ResultSet y false en caso contrario.

boolean next (): desplaza el puntero de registros sobre el registro que sigue al registro actual. Este método devuelve true si el puntero está en un registro válido y false en caso contrario (EOF).

- **boolean previous ()**: desplaza el puntero de registros sobre el registro anterior al registro actual. Este método devuelve true si el puntero está sobre un registro válido y false en caso contrario (BOF).

Posicionamiento en un ResultSet

Para todos estos métodos, excepto para el método `next`, es absolutamente necesario que el `ResultSet` sea de tipo `SCROLL_SENSITIVE` O `SCROLL_INSENSITIVE`. Si el `ResultSet` es de tipo `FORWARD_ONLY`, solo el método `next` funciona y en este caso los demás métodos producen una excepción. Los métodos siguientes permiten comprobar la posición del puntero de registros:

- **boolean isBeforeFirst()**: devuelve true si el puntero está situado antes del primer registro (BOF).
- **boolean isAfterLast()**: devuelve true si el puntero está situado después del último registro (EOF).
- **boolean isFirst()**: devuelve true si el puntero está situado en el primer registro.
- **boolean isLast()**: devuelve true si el puntero está situado en el último registro.
- **int getRow()**: devuelve el número del registro sobre el cual se encuentra

2 Instrucciones con el objeto tipo PreparedStatement

```
import java.sql.*;
public class TestPositionPunteroResultSet {
    public static void main(String[] args) {
        try (Connection cnx = getConnexion();
            Statement stm = cnx.createStatement(ResultSet.TYPE_SCROLL_SENSITIVE,
ResultSet.CONCUR_UPDATABLE)) {
            // Ejecutar la consulta y obtener el resultado
            ResultSet rs = stm.executeQuery("select * from PERSONAS");
            mostrarPosicionPuntero(rs);
            // Mientras hay registros
            while (rs.next()) {
                mostrarPosicionPuntero(rs);
            }
            mostrarPosicionPuntero(rs);
        } catch (SQLException e) {
            e.printStackTrace();
        }
    }
    private static void mostrarPosicionPuntero(ResultSet rs) {
        try {
            if (rs.isBeforeFirst()) {
                System.out.println("el puntero está antes del primer registro");
            }
            if (rs.isAfterLast()) {
                System.out.println("el puntero está después del último registro");
            }
            if (rs.isFirst()) {
                System.out.println("el puntero está en el primer registro");
            }
            if (rs.isLast()) {
                System.out.println("el puntero está en el último registro");
            }
            int position;
            position = rs.getRow();
            if (position != 0) {
                System.out.println("es el registro número " + position);
            }
        } catch (SQLException e) {
            // TODO Bloque catch autogenerated
            e.printStackTrace();
        }
    }
    private static Connection getConnexion() throws SQLException {
        String url = "jdbc:mysql://localhost/pruebasjava";
        String user = "root";
        String password = "root";
        return DriverManager.getConnection(url, user, password);
    }
}
```

el puntero está antes del primer registro
el puntero está en el primer registro
es el registro número 1
el puntero está en el último registro
es el registro número 2
el puntero está después del último registro

6.3.2024

Lectura de los datos en un ResultSet

El objeto Result Set facilita numerosos métodos que permiten la lectura de los campos de un registro. Cada uno de estos métodos es específico de un 15 de datos SQL. Por supuesto, hay que elegir adecuadamente el método de lectura según el tipo de datos del campo cuyo valor se quiere obtener. Sin embargo, algunos de estos métodos son relativamente flexibles y permiten la lectura de varios tipos de datos.

getInt- INTEGER | GetDouble – FLOAT DOUBLE | getString- CHAR VARCHAR
getObject – STRUCT JAVA OBJECT

Cada uno de estos métodos se presenta bajo dos formas. La primera recibe como argumento el número de la columna de la que se quiere obtener el valor. La numeración empieza con el 1. La segunda versión recibe una cadena de caracteres que representa el nombre de la columna en la base de datos. Si la petición utilizada para crear el ResultSet contiene alias, entonces las columnas llevan el nombre del alias y no el nombre del campo en la base de datos. Para una mejor legibilidad del código, es preferible usar los nombres de las columnas en vez de sus índices. Cuando en la base de datos un campo no contiene valores (NULL), los métodos devuelven un valor igual a 0 para los campos numéricos, un valor false para los campos booleanos y un valor null para los demás tipos. En algunos casos, podrían existir ciertas dudas relativas al valor realmente almacenado en la base de datos.

Lectura de los datos en un ResultSet

Por ejemplo, el método `getInt` puede devolver un valor igual a cero porque está realmente almacenado este valor en la base de datos o porque este campo admite valores nulos en la base de datos. Para resolver la duda, el método `wasNull` devuelve un booleano igual a `true` si el campo donde se hizo la última operación de lectura sobre el `ResultSet` contenía efectivamente un valor `null`.

El siguiente código resalta la manipulación de los tipos Java y SQL al escribir y leer información. Se enfatiza la manipulación de fechas.

Para escribir una fecha en una consulta parametrizada, es posible usar el método `setDate`, que espera en el segundo parámetro un objeto de tipo `java.sql.Date`. Un objeto de este tipo no debe manipularse en el resto de la aplicación. Es preferible utilizar un objeto de tipo `LocalDate` (si la parte horaria no es útil) o un objeto de tipo `LocalTime`. El método `java.sql.Date.valueOf` espera como parámetro un objeto de tipo `LocalDate` para obtener un objeto de tipo `java.sql.Date`.

Lectura de los datos en un ResultSet

Para escribir una fecha en una consulta parametrizada, también es posible usar el método `setObject`, que acepta un objeto como segundo parámetro. Este objeto puede ser de tipo `LocalDate`. El controlador JDBC podrá usar este objeto para guardarlo en una columna de tipo `Date`.

Para leer los datos, se adaptan los métodos `getXxx` de la interfaz `ResultSet`. La lectura de una fecha se realiza con el método `getDate`. Devuelve un objeto de tipo `java.sql.Date`.

Es posible obtener un objeto de tipo `LocalDate` a partir de un objeto de `java.sql.Date` utilizando el método `toLocalDate`.

ANÁLISIS Y PROGRAMACIÓN EN JAVA - IFCD004PO

```
import java.sql.*;
import java.time.*;
import java.time.format.*;
import java.util.TimeZone;
public class TestMappingTypeJavaSQL {
    public static void main(String[] args) {
        try (Connection cnx = getConnexion());
            PreparedStatement pstmt = cnx.prepareStatement("insert into PERSONAS(nombre, apellido,
fechadenacimiento) values(?,?,?)");
            Statement stm = cnx.createStatement()) {
                insererPersonas(pstmt);
                leerPersonas(stm);
            } catch (SQLException e) {
                e.printStackTrace();
            }
    }
    private static void insererPersonas(PreparedStatement pstmt) throws SQLException {
        //La siguiente instrucción permite hacer coincidir
        //la timezona de la JVM con la timezona del servidor MySQL.
        TimeZone.setDefault(TimeZone.getTimeZone("UTC"));
        //Agregar una primera persona Aplicar los parámetros
        pstmt.setString(1, "Eastwood");
        pstmt.setString(2, "Clint");
        //Escritura de la fecha de nacimiento
        //pstmt.setObject(3, LocalDate.of(1930, 5, 31));
        pstmt.setDate(3, Date.valueOf(LocalDate.of(1930, 5, 31)));
        //Ejecutar la consulta
        int resultado = pstmt.executeUpdate();
        if(resultado==1){
            System.out.println("La inserción ha tenido éxito");
        }
        //Agregar una segunda persona Aplicar los parámetros
        pstmt.setString(1, "Tarantino");
        pstmt.setString(2, "Quentin");
        pstmt.setObject(3, null);
        //Ejecutar la consulta
        resultado = pstmt.executeUpdate();
        if(resultado==1){
            System.out.println("La inserción ha tenido éxito");
        }
    }
}
```

6.3.2024

ANÁLISIS Y PROGRAMACIÓN EN JAVA - IFCD004PO

```
private static void leerPersonas(Statement stm) throws SQLException {
    ResultSet rs = stm.executeQuery("select id, nombre, apellido, fechadenacimiento from PERSONAS");
    System.out.println("id\t\tnombre\t\tapellido\t\tfecha de nacimiento");
    while(rs.next()){
        int id = rs.getInt("id");
        System.out.print(rs.isNull()?"desconocido":id);
        System.out.print("\t\t");
        String nombre = rs.getString("nombre");
        System.out.print(rs.isNull()?"desconocido":nombre);
        System.out.print("\t\t");
        String apellido = rs.getString("apellido");
        System.out.print(rs.isNull()?"desconocido":apellido);
        System.out.print("\t\t");
        Date sqlDate = rs.getDate("fechadeNacimiento");
        System.out.println(rs.isNull()?"desconocida":DateTimeFormatter.ofPattern("dd/MM/yyyy").format(sqlDate.toLocalDate()));
    }
}

private static Connection getConnexion() throws SQLException {
    String url = "jdbc:mysql://localhost/pruebasjava";
    String user = "root";
    String password = "root";
    return DriverManager.getConnection(url, user, password);
}
```

La inserción ha tenido éxito			
La inserción ha tenido éxito			
id	nombre	apellido	fecha de nacimiento
1	Juan	Ruperez	desconocida
2	Pablo	Marmol	desconocida
7	Eastwood	Clint	31/05/1930
8	Tarantino	Quentin	desconocida

6.3.2024

Enrique Krause Buedo

Modificación de los datos en un ResultSet

La modificación de los datos se efectúa simplemente utilizando los métodos `updateXxx`, donde `Xxx` corresponde al tipo de datos de la columna que se ha de actualizar. Como para el caso de los métodos `getXxx`, estos disponen de dos versiones: una recibe como argumento el índice de la columna que se ha de actualizar, la segunda recibe como argumento una cadena de caracteres que representa el nombre de la columna en la base de datos.

El tipo del segundo argumento esperado por estos métodos corresponde, lógicamente, al tipo de datos que hay que actualizar en el `ResultSet`. A continuación, se deben validar las modificaciones con el método `updateRow` o cancelar con el método `cancelRowUpdates`. El `ResultSet` debe ser obligatoriamente de tipo `CONCUR_UPDATABLE` para poder ser modificado. En caso contrario, la ejecución de estos métodos genera una excepción.

El siguiente ejemplo actualiza los nombres de las personas que usan `ResultSet`.

ANÁLISIS Y PROGRAMACIÓN EN JAVA - IFCD004PO

```
import java.sql.*;
import java.time.format.DateTimeFormatter;
public class TestModificationViaResultSet {
    public static void main(String[] args) {
        try (Connection cnx = getConnexion();
            Statement stmModif = cnx.createStatement(ResultSet.TYPE_SCROLL_SENSITIVE,
                ResultSet.CONCUR_UPDATABLE);
            Statement stmReading = cnx.createStatement()) {
            leerYModificarPersonas(stmModif);
            leerPersonas(stmReading);
        } catch (SQLException e) {
            e.printStackTrace();
        }
    }
    private static void leerYModificarPersonas(Statement stmModif) throws SQLException {
        ResultSet rs = stmModif.executeQuery("select id, nombre, apellido, fechadeNacimiento from PERSONAS");
        while(rs.next()){
            //Modificación del nombre poniéndolo en mayúsculas
            rs.updateString("nombre", rs.getString("nombre").toUpperCase());
            rs.updateRow();
        }
    }
    private static void leerPersonas(Statement stm) throws SQLException {
        ResultSet rs = stm.executeQuery("select id, nombre, apellido, fechadeNacimiento from PERSONAS");
        System.out.println("id\t\tnombre\t\tapellido\t\tfecha de nacimiento");
        while(rs.next()){
            int id = rs.getInt("id");
            System.out.print(rs.isNull()?"desconocido":id);
            System.out.print("\t\t");
            String nombre = rs.getString("nombre");
            System.out.print(rs.isNull()?"desconocido":nombre);
            System.out.print("\t\t");
            String apellido = rs.getString("apellido");
            System.out.print(rs.isNull()?"desconocido":apellido);
            System.out.print("\t\t");
            Date sqlDate = rs.getDate("fechadeNacimiento");
            System.out.println(rs.isNull()?"desconocida":DateTimeFormatter.ofPattern("dd/MM/yyyy").format(sqlDate.toLocalDate()));
        }
    }
    private static Connection getConnexion() throws SQLException {
        String url = "jdbc:mysql://localhost/pruebasjava";
        String user = "root";
        String password = "root";
        return DriverManager.getConnection(url, user, password);
    }
}
```

Supresión de datos en un ResultSet

La supresión de una línea se realiza de manera sencilla al situar el puntero sobre la línea que se ha de suprimir y llamando, a continuación al método `deleteRow`. La línea se elimina inmediatamente del `ResultSet` y de la base de datos. La posición del puntero de registros después del borrado depende del driver usado. Algunos drivers desplazan el puntero al registro siguiente, otros lo desplazan al anterior, e incluso algunos no modifican la posición del puntero. En este caso, hay que utilizar alguno de los métodos de desplazamiento para situar el puntero sobre un registro utilizable. El `ResultSet` debe ser obligatoriamente de tipo `CONCUR_UPDATABLE` de modo que sea posible suprimir datos en él. En caso contrario, la ejecución de un método `deleteRow` produce una excepción.

ANÁLISIS Y PROGRAMACIÓN EN JAVA - IFCD004PO

```
import java.sql.*;
import java.time.format.DateTimeFormatter;
public class TestSuppressionViaResultSet {
    public static void main(String[] args) {
        try (Connection cnx = getConnexion();
            Statement stmDelete = cnx.createStatement(ResultSet.TYPE_SCROLL_SENSITIVE,
                ResultSet.CONCUR_UPDATABLE);
            Statement stmReading = cnx.createStatement()) {
            leerYEliminarPersonas(stmDelete);
            leerPersonas(stmReading);
        } catch (SQLException e) {
            e.printStackTrace();
        }
    }
    private static void leerYEliminarPersonas(Statement stmDelete) throws SQLException {
        ResultSet rs = stmDelete.executeQuery("select id, nombre, apellido, fechadeNacimiento from PERSONAS");
        while(rs.next()){
            //Delete el registro si la personas se llama Eastwood
            System.out.println("Persona actual : " + rs.getString("nombre"));
            if("Eastwood".equalsIgnoreCase(rs.getString("nombre"))){
                System.out.println("Elimino la persona");
                rs.deleteRow();
            }
        }
    }
    private static void leerPersonas(Statement stm) throws SQLException {
        ResultSet rs = stm.executeQuery("select id, nombre, apellido, fechadeNacimiento from PERSONAS");
        System.out.println("id\t\tnombre\t\tapellido\t\tfecha de nacimiento");
        while(rs.next()){
            int id = rs.getInt("id");
            System.out.print(rs.isNull()?"desconocido":id);
            System.out.print("\t\t");
            String nombre = rs.getString("nombre");
            System.out.print(rs.isNull()?"desconocido":nombre);
            System.out.print("\t\t");
            String apellido = rs.getString("apellido");
            System.out.print(rs.isNull()?"desconocido":apellido);
            System.out.print("\t\t");
            Date sqlDate = rs.getDate("fechadeNacimiento");
            System.out.println(rs.isNull()?"desconocida":DateTimeFormatter.ofPattern("dd/MM/yyyy").format(sqlDate.toLocalDate()));
        }
    }
    private static Connection getConnexion() throws SQLException {
        String url = "jdbc:mysql://localhost/pruebasjava";
        String user = "root";
        String password = "root";
        return DriverManager.getConnection(url, user, password);
    }
}
```

6.3.2024

Enrique Krause Buedo

Inserción de datos en un ResultSet

Cada objeto ResultSet contiene una línea especial destinada a la inserción de datos. El puntero de registro se debe situar, previamente, en esta línea especial con la instrucción `moveToInsertRow`. Una vez situado el puntero sobre esta línea, es posible actualizarla mediante los métodos `updateXXX`. A continuación se debe validar la inserción de la línea mediante `insertRow`. Este método provoca la actualización de la base de datos. La línea de inserción se convierte en este momento en una línea normal del ResultsSet y el puntero de registro se posiciona en esta línea. Puede regresar a la línea en la que estaba antes de la inserción gracias al método `moveToCurrentRow`. Si no proporcionamos valores para todas las columnas, se insertarán valores null en la base de datos para las columnas que acepten valores null. La base de datos debe aceptar los valores nulos en estos campos porque, en caso contrario, se produce una excepción. El ResultSet debe ser obligatoriamente de tipo `CONCUR_UPDATABLE` si se quiere insertar datos en él. En caso contrario, la ejecución `moveToInsertRow` genera una excepción.

ANÁLISIS Y PROGRAMACIÓN EN JAVA - IFCD004PO

```
import java.sql.*;
import java.time.format.DateTimeFormatter;
public class TestInsertionViaResultSet {
    public static void main(String[] args) {
        try (Connection cnx = getConnexion();
            Statement stmInsertion = cnx.createStatement(ResultSet.TYPE_SCROLL_SENSITIVE,
                ResultSet.CONCUR_UPDATABLE);
            Statement stmReading = cnx.createStatement()) {
            agregarPersona(stmInsertion);
            leerPersonas(stmReading);
        } catch (SQLException e) {
            e.printStackTrace();
        }
    }
    private static void agregarPersona(Statement stmInsertion) throws SQLException {
        ResultSet rs = stmInsertion.executeQuery("select id, nombre, apellido, fechadeNacimiento from PERSONAS");
        rs.moveToInsertRow();
        rs.updateString("nombre", "BESSON");
        rs.updateString("apellido", "Luc");
        rs.insertRow();
    }
    private static void leerPersonas(Statement stm) throws SQLException {
        ResultSet rs = stm.executeQuery("select id, nombre, apellido, fechadeNacimiento from PERSONAS");
        System.out.println("id\t\tnombre\t\tapellido\t\tfecha de nacimiento");
        while(rs.next()){
            int id = rs.getInt("id");
            System.out.print(rs.isNull()?"desconocido":id);
            System.out.print("\t\t");
            String nombre = rs.getString("nombre");
            System.out.print(rs.isNull()?"desconocido":nombre);
            System.out.print("\t\t");
            String apellido = rs.getString("apellido");
            System.out.print(rs.isNull()?"desconocido":apellido);
            System.out.print("\t\t");
            Date sqlDate = rs.getDate("fechadeNacimiento");
            System.out.println(rs.isNull()?"desconocida":DateTimeFormatter.ofPattern("dd/MM/yyyy").format(sqlDate.toLocalDate()));
        }
    }
    private static Connection getConnexion() throws SQLException {
        String url = "jdbc:mysql://localhost/pruebasjava";
        String user = "root";
        String password = "root";
        return DriverManager.getConnection(url, user, password);
    }
}
```

6.3.2024

Enrique Krause Buedo